

GHAS ↔ Jira MCP Server

POC Requirements & Feature Specification

Date: April 16, 2026

Project: Closed-Loop Vulnerability Management POC

Repository: sautalwar/ghcopilot-pii-demo

Stakeholder: Saurabh Altalwar

Implementation Approach: Hybrid MCP Server + Express App (Phase 1) → GitHub App (Phase 2)

Contents

1. Executive Summary
2. Implementation Approach
3. Jira Integration & Closed-Loop Workflow
4. Real-Time CVE Feed
5. Zero-Day Intelligence & Early Disclosure
6. Exploitability Insights & Code Impact Analysis
7. Full Dependency Tree & Transitive Dependencies
8. Vulnerability Trends & Progress Tracking
9. Risk Exception Controls
10. Governance Checks Engine
11. License Compliance & SBOM
12. MCP Server Tools (Complete Registry)
13. Demo Flow (6-Act Structure)
14. Success Criteria
15. Architecture & File Layout

1. Executive Summary

This document captures all stakeholder requirements for a comprehensive GitHub Advanced Security (GHAS) POC that demonstrates closed-loop vulnerability management — from detection to remediation to compliance certification. The POC is designed to match and exceed Snyk's capabilities, with native Jira integration, real-time CVE intelligence, zero-day early warning, automated Copilot remediation, governance gates, license compliance, and executive-level vulnerability trend reporting.

The system handles the complete lifecycle:

- **Detect** → GHAS finds vulnerabilities (secrets, CVEs, CodeQL findings)
- **Deduplicate** → Search Jira before creating tickets (no duplicate bugs)
- **Track** → Create Jira ticket or add comment to existing one
- **Assign** → Route to GitHub Copilot (@copilot) for auto-fix
- **Remediate** → Copilot fixes code, opens PR
- **Gate** → Governance checks + license compliance must pass before merge
- **Close** → Jira ticket auto-closed when vulnerability resolved
- **Escalate** → If Copilot can't fix, flag for human review
- **Report** → Trends dashboard shows reduction over time

2. Implementation Approach

Phase 1: Hybrid MCP Server + Express App

The POC uses a hybrid architecture with two complementary surfaces from one shared codebase:

Layer	Technology	Audience	Purpose
MCP Server	@modelcontextprotocol /sdk (stdio)	Copilot / AI agents	Exposes tools: Jira ops, CVE lookups, zero-day intel, governance, licenses
Express App	Express.js (port 3000)	Humans (devs, security, CISOs)	Dashboards: CVE ticker, trends, blast radius, Jira timeline, audit trail
Shared Services	TypeScript in src/services/	Both layers	Core logic: all business rules, data sources, caching
GitHub Actions	Workflow YAML	Automation	Event-driven: GHAS alert → Jira, PR gate, remediation → close

Why Hybrid: MCP = Copilot-native (one config line per dev). Express = universal (browser access for security teams). Shared services = no fragmentation. MCP is GitHub's endorsed extensibility model — future-proof.

Phase 2: GitHub App / Extension (Separate Task)

A proper GitHub App installed at org level: webhook-driven automation, OAuth auth, Marketplace distribution. Reuses all Phase 1 services — it's a new surface on the same engine.

3. Jira Integration & Closed-Loop Workflow

Closed-loop integration between GHAS and Jira, matching Snyk's native Jira capability. Dual-mode: real Jira Cloud API or SQLite mock for demos.

MCP Tools:

Tool	Description
jira_search_issues	Search Jira by JQL (vuln type, file path, CVE ID)
jira_create_issue	Create bug with severity, file path, CVE details
jira_add_comment	Add finding comment to existing ticket
jira_transition_issue	Move ticket: Open → In Progress → Done
jira_get_issue	Get full issue details by key
jira_link_github	Add GitHub PR/issue link to Jira ticket

Deduplication Logic:

- Secret scanning → search by summary ~ "secret-leak" AND description ~ "{file_path}"
- Dependabot/CVE → search by summary ~ "{CVE-ID}" OR description ~ "{package_name}"
- CodeQL → search by summary ~ "{rule_id}" AND description ~ "{file_path}"
- Returns: { found, issueKey, confidence } — threshold configurable (default: 0.7)

Workflows:

- **Bridge Workflow** (ghas-jira-bridge.yml) — Triggered on code_scanning_alert, secret_scanning_alert, dependabot_alert, workflow_dispatch. Creates/updates Jira ticket, assigns @copilot.
- **Closer Workflow** (ghas-jira-closer.yml) — Triggered when alert fixed or PR merged. Transitions Jira ticket to Done with resolution comment.

4. Real-Time CVE Feed

Aggregates CVE data from two sources and cross-references against the repo's actual dependencies.

Source	API	Auth	Rate Limit
GitHub Advisory DB	GraphQL (api.github.com/graphql)	GITHUB_TOKEN (existing)	5000 req/hr
NVD (NIST)	REST v2 (services.nvd.nist.gov)	Optional API key	5/30s free, 50/30s with key

- **getLatestCVEs(hours?)** — Fetch CVEs published/modified in last N hours (default: 72)
- **getRepoVulnerabilities()** — Cross-reference CVEs against package.json/package-lock.json deps
- **getCVEDetails(cveId)** — Full details: CVSS score, vector, references, affected versions
- **streamCVEs()** — SSE endpoint for real-time push to frontend (no WebSocket needed)
- **Dependency Matching:** Enriches each CVE with: affectsThisRepo, installedVersion, patchedVersion, severity, exploitAvailable
- **Caching:** 5-min TTL for NVD, 1-min for GitHub Advisory DB, configurable via CVE_CACHE_TTL_SECONDS

5. Zero-Day Intelligence & Early Disclosure

Proactive vulnerability research that surfaces emerging threats BEFORE they have CVE IDs. This is the key differentiator vs Snyk.

Source	URL	What It Provides
GHSA (GitHub)	GraphQL API	Curator-reviewed advisories, often pre-NVD
CISA KEV	cisa.gov/.../known_exploited_vulnerabilities.js on	Actively exploited vulns — federal mandate
EPSS (FIRST.org)	api.first.org/data/v1/epss	30-day exploit probability score (0.0–1.0)
OSV.dev (Google)	api.osv.dev/v1/query	Cross-ecosystem DB, often pre-NVD entries
Dependabot Alerts	GitHub REST API	Repo-specific alerts with auto-fix suggestions

- **getZeroDayAlerts()** — Vulns with EPSS > 0.7 or on CISA KEV, not yet widely patched
- **getEarlyDisclosures(hours?)** — GHSA/OSV advisories without NVD entries yet
- **getExploitabilityScore(cveId)** — Composite: EPSS + CISA KEV + exploit-db references
- **getActiveThreats()** — Vulns with known exploits targeting packages in our dep tree

6. Exploitability Insights & Code Impact Analysis

Each CVE gets enriched with a composite exploitability profile showing real business impact.

Exploitability Profile Model:

Field	Description
epssScore	0.0–1.0 probability of exploitation in 30 days
cisaKey	Is it on the CISA Known Exploited Vulnerabilities list?
exploitMaturity	poc weaponized active-campaign unknown
affectedFunctions	Functions in THIS repo that use the vulnerable code path
affectedFiles	Files that import the vulnerable package
callChainDepth	Depth in dependency tree (1=direct, 2+=transitive)
impactIfUnpatched	Risk level, exposure window (days), attack vector, data at risk
impactIfPatched	Risk reduction %, breaking change risk, patch available, auto-fix possible

Code Impact Analysis:

- Parse package-lock.json for full transitive chain: direct dep → transitive → vulnerable package
- Static analysis (regex + AST-lite) of require()/import statements to find affected files
- Map affected functions to files → "These 4 files use lodash.template() which is vulnerable"
- Compute blast radius: % of codebase affected if vuln is exploited
- Remediation forecast: "If we fix: 95% risk reduction. If we don't: exploit in 12 days, 23% exposed"

7. Full Dependency Tree & Transitive Dependencies

Full dependency tree visualization with depth analysis, showing the complete chain from root to vulnerable transitive dependency — matching Snyk's tree view.

- Read package.json (direct deps) + package-lock.json (full resolved tree)
- Build tree structure showing: root → direct dep → transitive dep → ... → vulnerable package
- Depth indicator: how many levels deep is the vulnerable dependency?
- Highlight vulnerable paths in the tree (red edges)
- Show which direct dependency pulls in the vulnerable transitive dep
- Frontend: collapsible tree diagram with depth indicators and severity coloring

8. Vulnerability Trends & Progress Tracking

Tracks vulnerability lifecycle events over time to show reduction trends and remediation velocity. Uses SQLite event log (data/vuln-trends.db) seeded from GitHub APIs + synthetic demo data.

Feature	Description
getVulnTimeline(days?)	Time-series: open vs fixed vulns per day/week over N days
getMTTR()	Mean Time To Remediate — avg time from alert opened → fixed
getFixedVulns(days?)	Vulnerabilities fixed in last N days with details
getOpenByAge()	Open vulns bucketed: <7d, 7-30d, 30-90d, >90d
getSeverityTrends(days?)	Severity breakdown over time (critical trending down?)
getRemediationRate()	% fixed vs total, rolling 30/60/90 day windows
getCopilotImpact()	Vulns fixed by @copilot vs human — AI remediation contribution
seedFromGitHub()	Backfill historical data from Dependabot/code scanning/secret scanning APIs

Demo data: On first run, generates synthetic 90-day history with compelling downward trend. Flagged as [DEMO DATA] in UI. Real GitHub API data replaces it when available.

9. Risk Exception Controls

Policy-as-code engine via **.ghas-policy.yml** that defines which repos, files, directories, and rules are exempt from security monitoring due to pre-approved risk exceptions.

Exception Types:

- **Repo-level** — Entire repos excluded (e.g., legacy tools scheduled for decommission)
- **Directory-level** — Specific dirs excluded (e.g., test fixtures, demo incident files)
- **File-level** — Individual files excluded (e.g., dev-only config)
- **Rule-level** — Specific vuln rules excluded for specific scopes (e.g., CWE-798 in test/**)

Required Fields per Exception:

- **reason** — Why this exception exists
- **approved_by** — Who approved it (email or team)
- **expires** — Expiration date (or permanent: true)
- **jira_ticket** — (optional) Tracking ticket for the exception approval

Expiration policy: Warn 30 days before expiry. On expire: reactivate monitoring (configurable). Audit trail tracks every exception usage — detect overuse.

10. Governance Checks Engine

Mandatory checks via **.ghas-governance.yml** that must pass before code is certified as having gone through security compliance review. Runs as a PR gate.

Check	Description	Failure Behavior
Vulnerability Threshold	0 critical, 0 high allowed (configurable)	Block PR merge
Secret Scanning	No secrets detected in changed files	Block PR merge
License Compliance	All deps use approved licenses	Block PR merge
Dependency Audit	No critical/high dep vulns, lockfile present	Block PR merge
CodeQL Required	CodeQL ran with 0 errors, ≤5 warnings	Block PR merge
Security Review	Required for high-risk paths (security/, workflows/)	Await reviewer approval

- **runAllChecks(prNumber)** — Execute all enabled checks, return pass/fail report
- **getCertificationReport(prNumber)** — Full audit: checks ran, results, who approved, timestamp
- **isSecurityCertified(prNumber)** — Boolean: did PR pass all required checks?
- **getAuditTrail(days?)** — Historical records for compliance reporting (1-year retention)
- **GitHub Actions gate** — governance-gate.yml posts status check: ■ passed or ■ N checks failed

11. License Compliance & SBOM

Mandatory license auditing ensuring all dependencies (direct + transitive) use approved licenses. Catches GPL contamination, unknown licenses, and commercial violations.

License Categories:

Category	Examples	Action
Allowed	MIT, Apache-2.0, BSD-2-Clause, BSD-3-Clause, ISC, 0BSD, CC0-1.0	Auto-approve
Denied	GPL-2.0, GPL-3.0, AGPL-3.0, SSPL-1.0, CC-BY-NC-*	Auto-fail, block PR
Review Required	LGPL-2.1, LGPL-3.0, MPL-2.0, EPL-1.0, CDDL-1.0	Flag for legal review
Unknown	No license field or unrecognized string	Configurable: fail/warn/ignore

Data Sources for License Detection:

- package-lock.json → node_modules/{pkg}/package.json → license field
- GitHub License API (GET /repos/{owner}/{repo}/license)
- NPM Registry API (GET https://registry.npmjs.org/{package}/{version})
- SPDX license database for normalization (e.g., "Apache 2.0" → "Apache-2.0")

SBOM Export: Generates Software Bill of Materials in SPDX JSON format — ready for compliance audits, SOC 2, and supply chain security requirements.

12. MCP Server Tools (Complete Registry)

All tools exposed by the MCP server for Copilot and AI agents:

Tool	Category	Description
jira_search_issues	Jira	Search by JQL
jira_create_issue	Jira	Create bug with severity, CVE details
jira_add_comment	Jira	Add comment to existing ticket
jira_transition_issue	Jira	Move ticket through workflow
jira_get_issue	Jira	Get full issue details
jira_link_github	Jira	Link GitHub PR/issue to Jira
risk_check_exception	Risk	Is this file/rule excepted?
risk_list_exceptions	Risk	List all active risk exceptions
risk_expiring_soon	Risk	Which exceptions expire soon?
governance_check_pr	Governance	Run all governance checks on a PR
governance_get_policy	Governance	What checks are required?
governance_certification_report	Governance	Generate cert report for PR
license_audit	License	Full license compliance audit
license_check_package	License	Is this package's license approved?
license_get_violations	License	List all license violations
license_sbom	License	Generate SPDX SBOM

13. Demo Flow (6-Act Structure)

Act 1: Real-Time CVE Awareness — *the 'wow' opener*

- Show live CVE ticker from GitHub Advisory DB + NVD
- Filter to 'Affects My Code' — highlights CVEs matching repo deps
- Drill into CVE: CVSS score, affected versions, patch version

Act 1b: Zero-Day Intelligence — *the differentiator*

- Show early disclosures without CVE IDs (from GHSA/OSV)
- EPSS probability gauge: '87% chance of exploit in 30 days'
- Code blast radius: which files/functions are at risk
- 'What If' cards: patch now vs do nothing comparison

Act 2: Detection → Jira Ticket — *the Snyk-killer*

- Click 'Create Jira Ticket' on a CVE → dedup search first
- No duplicate → creates VULN-001 in Jira
- Trigger again → finds existing ticket → adds comment (dedup!)

Act 3: Copilot Auto-Remediation — *the closer*

- GitHub issue created with copilot:fix label
- @copilot creates branch, fixes vuln, opens PR
- Merge → Jira ticket auto-closed with PR link

Act 4: Escalation — *the safety net*

- If Copilot can't fix → re-labeled needs-human-review
- Jira comment added: 'AI could not auto-fix, human review needed'

Act 5: Progress & Trends — *the executive closer*

- 90-day reduction chart: 'Critical vulns down 73%'
- MTTR: '4.2 days → 6 hours'
- Copilot impact: 'Fixed 76% of vulns autonomously'

Act 6: Governance & Compliance — *the enterprise closer*

- Security gate status: ■ CERTIFIED or ■ FAILED
- License compliance: flag GPL dependency violation
- Risk exceptions with expiry dates and approval chains
- SBOM download, certification report, full audit trail

14. Success Criteria

Tier 1: Must-Have (Demo Blockers)

#	Criterion	Verification
S1	Real-time CVE feed shows live vulnerabilities	Ticker populates within 5 seconds
S2	'Affects My Code' correctly flags CVEs matching deps	≥1 CVE shows badge with correct package
S3	Jira dedup: same vuln twice → 1 ticket, 2 comments	Run bridge twice for same CVE
S4	Full Copilot loop: vuln → Jira → PR → merge → close	Loop completes without manual steps
S5	Governance gate blocks PR with license violation	PR with GPL dep → check fails → blocked
S6	Risk exceptions: excepted paths not flagged	File in excepted dir → governance skips
S7	License report shows deps with licenses + violations	Report has ≥1 violation, ≥1 approved
S8	Vuln reduction trends show downward curve	90-day chart with improvement trend

Tier 2: Should-Have (Differentiators)

#	Criterion	Verification
S9	Zero-day early disclosure (pre-CVE advisories)	≥1 GHSA advisory without CVE ID
S10	EPSS scoring for active threats	EPSS score > 0 for ≥1 CVE
S11	Code blast radius (affected files/functions)	Returns affected files + import chain
S12	'What If' cards (patched vs unpatched risk)	Both impact scenarios populated
S13	Copilot Impact card (AI vs human fix ratio)	Shows percentage with data
S14	MTTR metric with trend direction	Hours value with improvement indicator
S15	Full transitive dep tree (3+ depth)	Shows ≥1 transitive dep at depth 3+
S16	SBOM export (valid SPDX JSON)	Valid SPDX document returned

Overall: Demo-ready when all Tier 1 (S1–S8) pass + ≥6 of 8 Tier 2 (S9–S16). Demo runs end-to-end in under 15 minutes, covering all 6 Acts.

15. Architecture & File Layout

File Layout:

File	Purpose
src/mcp-server/index.ts	MCP server entry point (stdio transport)
src/mcp-server/jira-client.ts	IJiraClient + Cloud + Mock implementations
src/mcp-server/dedup-service.ts	Dedup logic with JQL builder
src/mcp-server/tools/*.ts	All MCP tool handlers (jira, risk, governance, license)
src/services/cve-feed-service.ts	CVE aggregation (GitHub Advisory DB + NVD)
src/services/dep-inventory.ts	Dependency inventory + transitive tree
src/services/zero-day-service.ts	Zero-day intelligence (EPSS, CISA KEV, OSV)
src/services/exploitability.ts	Exploitability profiling + code impact
src/services/vuln-trends-service.ts	Vulnerability trends + MTTR
src/services/risk-exceptions-service.ts	Risk exception engine (.ghas-policy.yml)
src/services/governance-service.ts	Governance checks engine (.ghas-governance.yml)
src/services/license-compliance-service.ts	License auditing + SBOM
src/api/jira-routes.ts	Jira demo frontend routes
src/api/cve-routes.ts	CVE feed + SSE streaming routes
src/api/zero-day-routes.ts	Zero-day intelligence routes
src/api/vuln-trends-routes.ts	Trends dashboard routes
src/api/governance-routes.ts	Governance, exceptions, licenses, audit routes
.github/workflows/ghas-jira-bridge.yml	GHAS alert → Jira ticket
.github/workflows/ghas-jira-closer.yml	Remediation → close Jira
.github/workflows/governance-gate.yml	PR gate: governance checks
.ghas-policy.yml	Risk exception controls (policy-as-code)
.ghas-governance.yml	Governance checks + license compliance policy

New Dependencies:

- **@modelcontextprotocol/sdk** — MCP server SDK
- **js-yaml** — Parse policy YAML files
- **spdx-license-list** — License ID normalization
- node-fetch or built-in fetch (Node 18+) — for external API calls